

Individual Contribution Report — Megan Hickey

1. Assigned Features

My main contributions to the Gym Management System included:

- Workout class functionality
- Membership functionality
- Trainer and Member menu development
- Input validation and error handling
- Date and time formatting
- Application logging
- Admin file exports

Workout Classes

I worked with `WorkoutClass`, `WorkoutClassService`, `WorkoutClassDAO`, and the workout class functionality in the Trainer and Member menus.

My work included creating, retrieving, updating, and deleting workout classes, retrieving classes by trainer, and validating class information before database operations. I also implemented `DateTimeFormatter` and exception handling for class date and time input.

Memberships

I worked with `Membership`, `MembershipService`, `MembershipDAO`, and membership functionality in the Trainer and Member menus.

My work included purchasing memberships, retrieving memberships by user, calculating personal membership expenses, and calculating annual membership revenue.

Menus

I worked mainly on `TrainerMenu` and `MemberMenu`. I separated menu functionality into helper methods and added input validation to improve readability and prevent invalid input from crashing the application.

Logging

I implemented application logging using `java.util.logging` and a separate `AppLogger` class. The application records important events such as system startup, failed login attempts, database transaction errors, and Admin actions in `app.log`.

The generated log file is excluded from GitHub through `.gitignore`.

Admin File Export

I implemented the Admin File Export feature for:

- Merchandise Inventory Report
- Membership Revenue Report

Reports are written as `.txt` files inside the `/reports/` folder. The application checks for the folder and creates it automatically if it does not already exist.

2. GitHub Contributions

Development work was completed using separate Git branches and pull requests.

Branches: [Link to branches](#)

Pull Requests: [Link to my pull requests](#)

Commit History: [Link to my commits](#)

3. Challenges and Problem Solving

Date and Time Input

Incorrect date and time input could cause parsing errors. I used `DateTimeFormatter`, `DateTimeParseException`, and try/catch blocks to validate input and display an error instead of allowing the program to crash.

Working With Shared Files

Some features required changes to files that my teammate was also working on. We used separate Git branches and coordinated our changes to reduce merge conflicts.

4. Skills Learned

During this project I gained experience with:

- Java Model, Service, and DAO structure

- JDBC and PostgreSQL
 - `PreparedStatement`
 - BCrypt password security
 - Input validation and exception handling
 - `LocalDate`, `LocalTime`, and `DateTimeFormatter`
 - Git branches, commits, and pull requests
 - `java.util.logging`
 - Java File I/O
 - Creating directories and text files
-

5. Team Reflection

Dividing the project into separate features allowed us to work independently while contributing to the same application. Git branches helped keep our changes separate.

The main difficulty was coordinating changes to shared files. More frequent pushes and clearer ownership of shared files would make this process easier in future projects.

6. AI Usage Log

I used ChatGPT to:

- Explain DAO and service classes
- Troubleshoot Java and JDBC issues
- Learn `DateTimeFormatter`
- Understand exception handling
- Learn `java.util.logging`
- Learn Java File I/O
- Understand directory creation and `FileWriter`
- Review code structure
- Assist with project documentation

I used AI to help explain unfamiliar concepts, troubleshoot issues, and understand how to implement features in the project.

7. AI Feature Challenge Integration & Analysis

AI Dialogue

I used AI while learning how to implement the Admin File Export feature. My prompts focused on:

- Exporting application data to a text file
- Using `FileWriter`
- Closing file streams
- Using try-with-resources
- Creating the `/reports/` folder automatically

Technical Reflection

I learned how to retrieve existing application data and write it to a text file using `FileWriter`. I used try-with-resources so the writer closes automatically. Closing the writer releases resources and ensures buffered data is written to the file. If the program stops before the writer is properly closed, the report may be incomplete.

Code Showcase

```
public void exportReport() throws SQLException {

    createReportsFolder();

    ArrayList<Membership> memberships =
        membershipService.getAllMemberships();

    try (FileWriter writer =
        new FileWriter("reports/membership_revenue_report.txt")) {

        writer.write("Membership Revenue Report\n");
        writer.write("=====\n");

        double totalRevenue = 0;

        for (Membership membership : memberships) {

            writer.write("Membership ID: " + membership.getMembershipId() +
"\n");
            writer.write("User ID: " + membership.getUserId() + "\n");
            writer.write("Membership Type: " + membership.getMembershipType()
+ "\n");
            writer.write("Price: $" + membership.getPrice() + "\n");
            writer.write("Purchase Date: " + membership.getPurchaseDate() +
"\n");
            writer.write("-----\n");

            totalRevenue += membership.getPrice();

        }

        writer.write("\n");
        writer.write("Total Membership Revenue: $" + totalRevenue + "\n");

    } catch (IOException e) {
```

```
        System.out.println("Unable to create membership report.");  
    }  
}
```